LAB 20 - PROMPTS

Capstone: One Command Runs the Pipeline End to End, with an Agentic Layer and Human Approval

Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder. Work down the steps: copy each prompt into the assistant, review the code it writes, then run the command. The complete, working code is already in Lab20-agentic-capstone/solution/ if you want to compare. Never paste API keys into an AI chat.

Step 1: Build the agent layer (K1, A7)

Prompt - copy into your AI coding assistant

Create labs_fullcode/pipeline/agents.py. Agents: technical_analyst(symbol, bars) - EMA 20/50, RSI 14, MACD histogram and ATR 14 from pipeline.indicators -> score in [-1, 1]; news_analyst(cfg, symbol, headlines) - lexicon sentiment of pipeline.ingest.load_headlines (POSITIVE / NEGATIVE word sets); debate(cfg, tech, news) - bull and bear points and balance = 0.6 x technical + 0.4 x news; trader(symbol, tech, research, equity, held) - buy if balance > 0.25 and flat, sell if < -0.25 and held, confidence = min(1, |balance| + 0.3), qty = floor(equity x 0.25 x confidence / close); risk_manager(cfg, proposal, tech, equity) - DETERMINISTIC hard limits from config agents (allow-list, min_confidence, cap = min(equity x max_position_pct, max_order_value) with resizing, ATR bracket stop = close - stop_atr x ATR and target = close + target_atr x ATR). Optional LLM: only if the ANTHROPIC_API_KEY named in secrets_from_env is set and the anthropic package imports, news_analyst and debate call client.messages.create(model=cfg['agents']['model'] (claude-sonnet-5-5), max_tokens=1024, system prompt that treats headlines as untrusted data and asks for ONE JSON object, extra_headers={'anthropic-beta': 'server-side-fallback-2026-07-01'}, extra_body={'fallbacks': 'default'}), return None on refusal or any error so the rule-based brain is used. run(symbols, execute_orders, yes, reset, session) also applies pipeline.risk.RiskManager.check, asks a human to type YES (EOF = declined) before OrderManager.rebalance(..., units='shares', order_type='bracket' in the regular session), writes _outputs/Lab20_decision_trace.json and chart Lab20_agent_pipeline.png. main(argv) for `python -m pipeline agents --symbols --execute --yes --reset --session`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: python -m pipeline agents --reset prints three symbol blocks and 'Propose-only run'.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Build the end-to-end orchestrator (A7, A10)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/e2e.py. run(execute_orders=False, yes=False, fast=False) calls each stage's run() in order - check (reset), validate, ingest, hypothesis, indicators, reconcile, signals, execute (equal weights over the long universe targets from signals, capped at 20%), evaluate, search, ml, regime, abtest, pairs, backtest, runner.replay for 2025, webhook.selftest, diagnose, report - with contextlib.redirect_stdout into _outputs/Lab20_e2e_log.txt, timing each stage and printing one line: lab, stage, status (OK, GATE or ERROR), seconds and a short result. Exceptions are caught (ERROR) and the run continues. Gates: check not ready, ingest FAIL, diagnose BLOCK, report monitoring ALERT or kill switch on -> agents run propose-only. Last stage: agents.run; with --execute and gates passed ask the human for YES on the real console (or --yes for a scripted demo). Write _outputs/Lab20_e2e_summary.json and chart Lab20_e2e_pipeline.png (one coloured box per stage). --fast skips search, ml and pairs. Exit code 1 if any stage errored. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: python -m pipeline e2e --help shows --execute, --yes and --fast.

Step 3: Run the whole pipeline (A7)

Do this (no prompt needed)
Run: python -m pipeline e2e (same as: python3 Lab20-agentic-capstone/e2e_stage.py).

Check: 20 stages, all OK, in about 6-10 seconds offline: e.g. hypothesis 'p 0.044 -> SUPPORTED', reconcile 'USD 79,878', signals '7/9 universe targets long', execute '7 paper orders', ml 'NO EDGE', run '+16.25% vs buy & hold +17.06%', webhook '7/7', diagnose 'CLEAR for paper trading', report '5/5 monitoring OK', agents 'NVDA APPROVED, AAPL no trade, SPY APPROVED; approval not requested'; 'RESULT: 20 stages ..., 0 errors, gates passed'.

Step 4: Trace the run (A10)

Do this (no prompt needed)
Open _outputs/Lab20_e2e_log.txt (the full output of every stage), _outputs/Lab20_e2e_summary.json and _outputs/Lab20_e2e_pipeline.png.

Check: The log contains each stage's full printout in order; the chart shows 20 green boxes.

Step 5: Read the agents' decisions (K1, A10)

Do this (no prompt needed)
Run: python -m pipeline agents --reset and open _outputs/Lab20_decision_trace.json and _outputs/Lab20_agent_pipeline.png. Follow one symbol from evidence to decision.

Check: Rule-based on 2025-12-31 data: NVDA technical +0.75, news +0.25, BUY 114 resized to 107 shares (cap USD 20,000) with stop 175.5 / target 201.9 - APPROVED; AAPL HOLD (balance +0.02); SPY technical +0.72, news -0.33, BUY 22 - APPROVED with stop 664.75 / target 694.45.

Step 6: Apply human oversight (A7, K1)

Do this (no prompt needed)
Run: python -m pipeline e2e --execute. When asked, type YES only if you agree with the approved orders; any other answer declines them.

Check: With YES: 2 paper bracket orders (NVDA 107, SPY 22) and 'approval granted, 2 paper orders'; otherwise 'approval declined, 0 paper orders'.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 7: Prove the gates stop the orders (A7)

Do this (no prompt needed)
Run: python -m pipeline kill --on, then python -m pipeline e2e --fast --execute --yes, then python -m pipeline kill --off.

Check: 'gates failed ... agents run in propose-only mode', 0 paper orders, 'gates FAILED (no orders)' - even with --yes the kill switch wins.

Step 8: Optional - switch on the LLM agents (K1)

Do this (no prompt needed)
Set ANTHROPIC_API_KEY in .env (with the anthropic package installed) and rerun python -m pipeline agents --reset. The news and debate agents then use Claude (claude-sonnet-5-5); if a call fails or is refused the pipeline falls back to the rules. The risk manager and the pipeline risk checks never change.

Check: The banner shows 'Agent brain: LLM (claude-sonnet-5-5)' and each News line ends with (llm); orders still need the same limits and a human YES.

Step 9: Run the capstone on the live paper account (A7)

Do this (no prompt needed)
With Alpaca paper keys set, run python -m pipeline e2e --execute during class. Approved agent orders are sent as extended-hours limit orders (no bracket outside the regular session) - check them in the Alpaca paper dashboard, then flatten with python -m pipeline execute --live-paper --flatten.

Check: The check stage shows ONLINE and the real session; agent orders appear in the dashboard with client ids starting lcc-agent.

Step 10: Evaluate the pipeline (A10, K1)

Prompt - copy into your AI coding assistant
Using labs_fullcode/_outputs/Lab20_e2e_summary.json and labs_fullcode/_outputs/Lab20_decision_trace.json, draft three sentences for Lion City Capital's investment committee evaluating the end-to-end pipeline: the evidence from the stages, the risks of the agent layer, and the controls that keep it safe. Keep it factual and quote two numbers.

Open in Claude | **Open in ChatGPT**

Check: Three sentences that cite stage results (for example forward test +16.25% vs backtest +17.04%) and name the controls.

Run it

Step	Command / file
Run Lab 20	python3 Lab20-agentic-capstone/e2e_stage.py (same as: python -m pipeline e2e)
The complete code	pipeline/agents.py, pipeline/e2e.py (reference copy: Lab20-agentic-capstone/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.